

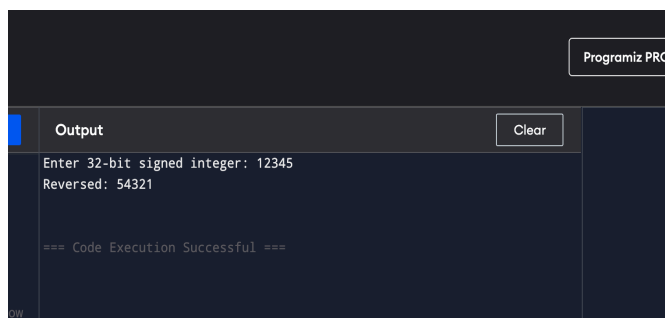
Q1 — Reverse a 32-bit signed integer (overflow-safe).

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int reverse_int(int x) {  
    long long rev = 0;  
    while (x != 0) {  
        int digit = x % 10;  
        rev = rev * 10 + digit;  
        if (rev > INT_MAX || rev < INT_MIN) return 0; // overflow  
        x /= 10;  
    }  
    return (int)rev;  
}
```

```
int main() {  
    int n;  
    printf("Enter 32-bit signed integer: ");  
    if (scanf("%d", &n) != 1) return 0;  
    int r = reverse_int(n);  
    if (r == 0) printf("Reversed value is 0 (possible overflow or original reversed to 0).\n");  
    else printf("Reversed: %d\n", r);  
    return 0;  
}
```



The screenshot shows a terminal window with a dark background. In the top right corner, there is a button labeled "Programiz PRO". Below this, there is a section titled "Output" with a "Clear" button next to it. The output text reads: "Enter 32-bit signed integer: 12345", "Reversed: 54321", and "=== Code Execution Successful ===".

Q2 — Check for a valid string

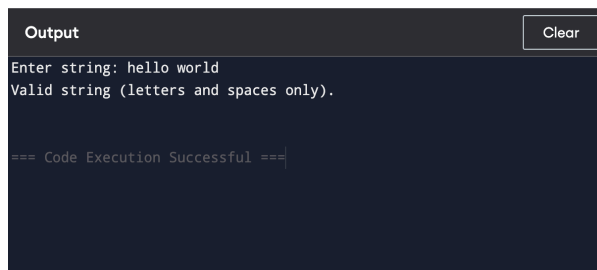
```
#include <stdio.h>

#include <ctype.h>

#include <string.h>

int is_valid_string(const char *s) {
    for (int i = 0; s[i]; ++i) {
        if (!(isalpha((unsigned char)s[i]) || isspace((unsigned char)s[i]))) return 0;
    }
    return 1;
}

int main() {
    char buf[1024];
    printf("Enter string: ");
    if (!fgets(buf, sizeof(buf), stdin)) return 0;
    // remove newline
    buf[strcspn(buf, "\n")] = 0;
    if (is_valid_string(buf)) printf("Valid string (letters and spaces only).\n");
    else printf("Invalid string (contains non-letters/non-space characters).\n");
    return 0;
}
```



```
Output
Enter string: hello world
Valid string (letters and spaces only).

=== Code Execution Successful ===
```

Q3) WRITE REG NO USING BINARY AND LINEAR SEARCH

```
#include <stdio.h>

int main() {
```

```
int arr[100], n, key, i, found = 0;

// Input number of registration numbers
printf("Enter the number of registration numbers: ");
scanf("%d", &n);

// Input registration numbers
printf("Enter %d registration numbers:\n", n);
for (i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
}
printf("Enter the registration number to search: ");
scanf("%d", &key);

// Linear Search
for (i = 0; i < n; i++) {
    if (arr[i] == key) {
        found = 1;
        break;
    }
}

// Display result
if (found)
    printf("Registration number %d found at position %d.\n", key, i + 1);
else
    printf("Registration number %d not found.\n", key);

return 0;
```

```
}
```

```
Output
Enter the number of registration numbers: 5
Enter 5 registration numbers:
101 202 303 404 505
Enter the registration number to search: 303
Registration number 303 found at position 3.

=== Code Execution Successful ===
```

BINARY SEARCH

```
#include <stdio.h>
```

```
int main() {
```

```
    int arr[100], n, key, low, high, mid, found = 0;
```

```
    // Input number of registration numbers
```

```
    printf("Enter the number of registration numbers: ");
```

```
    scanf("%d", &n);
```

```
    // Input registration numbers (sorted order)
```

```
    printf("Enter %d registration numbers in ascending order:\n", n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    // Input the registration number to search
```

```
    printf("Enter the registration number to search: ");
```

```
    scanf("%d", &key);
```

```
    // Binary Search
```

```
    low = 0;
```

```
    high = n - 1;
```

```

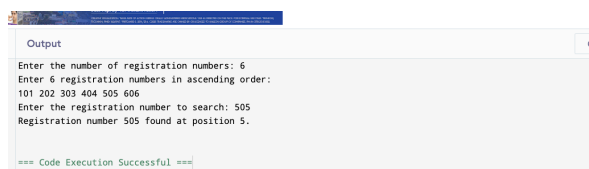
while (low <= high) {
    mid = (low + high) / 2;

    if (arr[mid] == key) {
        found = 1;
        break;
    } else if (arr[mid] < key) {
        low = mid + 1;
    } else {
        high = mid - 1;
    }
}

// Display result
if (found)
    printf("Registration number %d found at position %d.\n", key, mid + 1);
else
    printf("Registration number %d not found.\n", key);

return 0;
}

```



The screenshot shows a code execution environment with an 'Output' window. The output text is as follows:

```

Enter the number of registration numbers: 6
Enter 6 registration numbers in ascending order:
101 202 303 404 505 606
Enter the registration number to search: 505
Registration number 505 found at position 5.

=== Code Execution Successful ===

```

Q4— Identify location (index) of element in given array

```
#include <stdio.h>
```

```
int main() {  
    int n;  
    printf("Enter number of elements: ");  
    if (scanf("%d", &n) != 1 || n <= 0) return 0;  
    int a[n];  
    for (int i = 0; i < n; ++i) {  
        printf("a[%d]: ", i);  
        scanf("%d", &a[i]);  
    }  
    int key;  
    printf("Enter element to find: ");  
    scanf("%d", &key);  
    int found = 0;  
    for (int i = 0; i < n; ++i) {  
        if (a[i] == key) {  
            printf("Element %d found at index %d\n", key, i);  
            found = 1;  
            break; // remove break to print all occurrences  
        }  
    }  
    if (!found) printf("Element not found\n");  
    return 0;  
}
```

```
Output
Enter number of elements: 5
a[0]: 1
a[1]: 2
a[2]: 4
a[3]: 5
a[4]: 6
Enter element to find: 4
Element 4 found at index 2

=== Code Execution Successful ===
```

Q5— Given array print odd and even values

```
#include <stdio.h>
```

```
int main() {
    int n;

    printf("Number of elements: ");

    if (scanf("%d", &n) != 1 || n <= 0) return 0;

    int a[n];

    for (int i = 0; i < n; ++i) scanf("%d", &a[i]);

    printf("Even numbers: ");

    for (int i = 0; i < n; ++i) if (a[i] % 2 == 0) printf("%d ", a[i]);

    printf("\nOdd numbers: ");

    for (int i = 0; i < n; ++i) if (a[i] % 2 != 0) printf("%d ", a[i]);

    printf("\n");

    return 0;
}
```

```
Output
Number of elements: 5
2 3 4 5 6 7
Even numbers: 2 4 6
Odd numbers: 3 5

=== Code Execution Successful ===
```

Q6 — Sum of Fibonacci series (sum of first n Fibonacci numbers)

```
#include <stdio.h>

int main() {

    int n;

    printf("Enter number of Fibonacci terms (n >= 1): ");

    if (scanf("%d", &n) != 1 || n < 1) return 0;

    long long a = 0, b = 1;

    long long sum = 0;

    if (n >= 1) sum += a; // include F0 if desired; if you want starting at F1 change accordingly

    if (n >= 2) sum += b;

    for (int i = 3; i <= n; ++i) {

        long long c = a + b;

        sum += c;

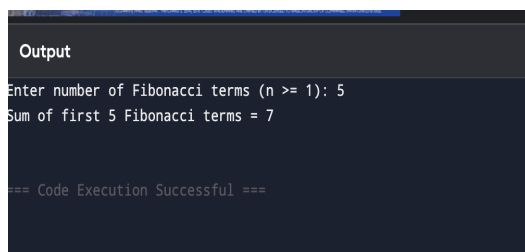
        a = b; b = c;

    }

    printf("Sum of first %d Fibonacci terms = %lld\n", n, sum);

    return 0;

}
```

A screenshot of a terminal window with a dark background. The title bar at the top says "Output". The terminal shows the program's execution: it prompts "Enter number of Fibonacci terms (n >= 1): 5", then outputs "Sum of first 5 Fibonacci terms = 7". At the bottom, it shows "=== Code Execution Successful ===".

```
Output
Enter number of Fibonacci terms (n >= 1): 5
Sum of first 5 Fibonacci terms = 7

=== Code Execution Successful ===
```

Q7 — Factorial of a number

```
#include <stdio.h>
```



```

unsigned long long factorial(unsigned int n) {

    unsigned long long res = 1;

    for (unsigned int i = 2; i <= n; ++i) res *= i;

    return res;

}

```

```

int main() {

    unsigned int n;

    printf("Enter non-negative integer: ");

    if (scanf("%u", &n) != 1) return 0;

    printf("%u! = %llu\n", n, factorial(n));

    return 0;

}

```



```

Output
Enter number of elements: 5 1 2 3 4 5
Enter elements (for binary search: enter sorted ascending array):
Enter key to search: 4
Linear search: found at index 3
Binary search: found at index 3

=== Code Execution Successful ===

```

Q8— Print the index of repeated characters given in an array

```
#include <stdio.h>
```

```
#include <string.h>
```

```

int main() {

    char s[1024];

    printf("Enter string: ");

    if (!fgets(s, sizeof(s), stdin)) return 0;

    s[strcspn(s, "\n")] = 0;

```

```

int len = strlen(s);
int printed[256] = {0};

for (int ch = 0; ch < 256; ++ch) {
    int count = 0;
    for (int i = 0; i < len; ++i) if ((unsigned char)s[i] == ch) ++count;
    if (count > 1) {
        printf("Character '%c' repeats at indices: ", (char)ch);
        for (int i = 0; i < len; ++i) {
            if ((unsigned char)s[i] == ch) printf("%d ", i);
        }
        printf("\n");
    }
}

return 0;
}

```

Output

Clear

```

Enter string: banana
Character 'a' repeats at indices: 1 3 5
Character 'n' repeats at indices: 2 4

=== Code Execution Successful ===

```

Q9 — Print the frequently repeated numbers count from an array

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

int cmp_int(const void *a, const void *b) {
    return (*(int*)a) - (*(int*)b);
}

```

```
int main() {
```

```

int n;

printf("Enter number of elements: ");

if (scanf("%d", &n) != 1 || n <= 0) return 0;

int *a = malloc(n * sizeof(int));

for (int i = 0; i < n; ++i) scanf("%d", &a[i]);


qsort(a, n, sizeof(int), cmp_int);


int i = 0;

int found_any = 0;

while (i < n) {

    int j = i + 1;

    while (j < n && a[j] == a[i]) ++j;

    int count = j - i;

    if (count > 1) {

        printf("Number %d repeats %d times\n", a[i], count);

        found_any = 1;

    }

    i = j;

}

if (!found_any) printf("No repeated numbers\n");

free(a);

return 0;

}

```

Output

Clear

```

Enter number of elements: 7
2 4 5 2 4 2 3
Number 2 repeats 3 times
Number 4 repeats 2 times

=== Code Execution Successful ===

```

Q10 — Sum of rows and columns in a 2D array

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int r, c;
```

```
    printf("Enter rows and columns: ");
```

```
    if (scanf("%d %d", &r, &c) != 2 || r <= 0 || c <= 0) return 0;
```

```
    int **mat = malloc(r * sizeof(int*));
```

```
    for (int i = 0; i < r; ++i) mat[i] = malloc(c * sizeof(int));
```

```
    printf("Enter matrix elements row-wise:\n");
```

```
    for (int i = 0; i < r; ++i)
```

```
        for (int j = 0; j < c; ++j)
```

```
            scanf("%d", &mat[i][j]);
```

```
    for (int i = 0; i < r; ++i) {
```

```
        long long sum = 0;
```

```
        for (int j = 0; j < c; ++j) sum += mat[i][j];
```

```
        printf("Sum of row %d = %lld\n", i, sum);
```

```
    }
```

```
    for (int j = 0; j < c; ++j) {
```

```
        long long sum = 0;
```

```

        for (int i = 0; i < r; ++i) sum += mat[i][j];

        printf("Sum of column %d = %lld\n", j, sum);

    }

    for (int i = 0; i < r; ++i) free(mat[i]);

    free(mat);

    return 0;

}

```



The screenshot shows a terminal window titled "Output" with a "Clear" button. The output text is as follows:

```

Enter rows and columns: 2 3
Enter matrix elements row-wise:
1 2 3
4 5 6
Sum of row 0 = 6
Sum of row 1 = 15
Sum of column 0 = 5
Sum of column 1 = 7
Sum of column 2 = 9

=== Code Execution Successful ===

```

Q11 — Check for elements repeated twice in an array

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int cmp_int(const void *a, const void *b) { return (*(int*)a) - (*(int*)b); }
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter number of elements: ");
```

```
    if (scanf("%d", &n) != 1 || n <= 0) return 0;
```

```
    int *a = malloc(n * sizeof(int));
```

```
    for (int i = 0; i < n; ++i) scanf("%d", &a[i]);
```

```
    qsort(a, n, sizeof(int), cmp_int);
```

```

int i = 0;
int found = 0;
while (i < n) {
    int j = i + 1;
    while (j < n && a[j] == a[i]) ++j;
    int count = j - i;
    if (count == 2) {
        printf("Element %d appears exactly twice\n", a[i]);
        found = 1;
    }
    i = j;
}
if (!found) printf("No element appears exactly twice\n");
free(a);
return 0;
}

```



The screenshot shows a dark-themed output window with a 'Clear' button in the top right corner. The text inside the window is as follows:

```

Output
Enter number of elements: 7
1 2 3 4 3 5
2 4

Element 2 appears exactly twice
Element 3 appears exactly twice

=== Code Execution Successful ===

```

Q12 — Given 2D matrix print largest element

```

#include <stdio.h>

#include <limits.h>

#include <stdlib.h>

```

```

int main() {

    int r, c;

```

```

printf("Enter rows and columns: ");
if (scanf("%d %d", &r, &c) != 2 || r <= 0 || c <= 0) return 0;
int maxv = INT_MIN;
printf("Enter matrix elements row-wise:\n");
for (int i = 0; i < r; ++i)
    for (int j = 0; j < c; ++j) {
        int v;
        scanf("%d", &v);
        if (v > maxv) maxv = v;
    }
printf("Largest element = %d\n", maxv);
return 0;
}

```

Output

Clear

```

Enter rows and columns: 2 2
Enter matrix elements row-wise:
4 9 1
7 5 3
Largest element = 9

=== Code Execution Successful ===

```

Q13. 2D add,sub,multiply,transverse

```
#include <stdio.h>
```

```

int main() {
    int A[10][10], B[10][10], C[10][10], T[10][10];
    int r, c, i, j, k;

    printf("Enter rows and columns: ");
    scanf("%d %d", &r, &c);

```

```
printf("Enter elements of matrix A:\n");
```

```
for (i = 0; i < r; i++)
```

```
    for (j = 0; j < c; j++)
```

```
        scanf("%d", &A[i][j]);
```

```
printf("Enter elements of matrix B:\n");
```

```
for (i = 0; i < r; i++)
```

```
    for (j = 0; j < c; j++)
```

```
        scanf("%d", &B[i][j]);
```

```
// Addition
```

```
printf("\nAddition of two matrices:\n");
```

```
for (i = 0; i < r; i++) {
```

```
    for (j = 0; j < c; j++) {
```

```
        C[i][j] = A[i][j] + B[i][j];
```

```
        printf("%d ", C[i][j]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
// Subtraction
```

```
printf("\nSubtraction of two matrices:\n");
```

```
for (i = 0; i < r; i++) {
```

```
    for (j = 0; j < c; j++) {
```

```
        C[i][j] = A[i][j] - B[i][j];
```

```
        printf("%d ", C[i][j]);
```

```
    }
```

```
    printf("\n");
```

```
}
```



```

// Multiplication

printf("\nMultiplication of two matrices:\n");

for (i = 0; i < r; i++)
    for (j = 0; j < c; j++) {
        C[i][j] = 0;
        for (k = 0; k < c; k++)
            C[i][j] += A[i][k] * B[k][j];
    }

for (i = 0; i < r; i++) {
    for (j = 0; j < c; j++)
        printf("%d ", C[i][j]);
    printf("\n");
}

// Transpose

printf("\nTranspose of Matrix A:\n");

for (i = 0; i < r; i++)
    for (j = 0; j < c; j++)
        T[j][i] = A[i][j];

for (i = 0; i < c; i++) {
    for (j = 0; j < r; j++)
        printf("%d ", T[i][j]);
    printf("\n");
}

return 0;

```

}

A screenshot of a C++ program's output window. The window has a title bar with 'Output' and a 'Clear' button. The output text is as follows:
Enter rows and columns: 2 2
Enter elements of matrix A:
1 2
3 4
Enter elements of matrix B:
5 6
7 8
Addition of two matrices:
6 8
10 12
Subtraction of two matrices:
-4 -4
-4 -4
Multiplication of two matrices:
19 22
43 50
Transpose of Matrix A:
1 3
2 4
At the bottom, it says 'Code Execution Successful :)'

15) 3D add,sub,multiply,transverse

```
#include <stdio.h>
```

```
int main() {
```

```
    int x, y, z;
```

```
    int A[10][10][10], B[10][10][10], C[10][10][10];
```

```
    int i, j, k;
```

```
    printf("Enter dimensions (x y z): ");
```

```
    scanf("%d %d %d", &x, &y, &z);
```

```
    printf("Enter elements of 3D Matrix A:\n");
```

```
    for (i = 0; i < x; i++)
```

```
        for (j = 0; j < y; j++)
```

```
            for (k = 0; k < z; k++)
```

```
                scanf("%d", &A[i][j][k]);
```

```
    printf("Enter elements of 3D Matrix B:\n");
```

```
    for (i = 0; i < x; i++)
```

```
        for (j = 0; j < y; j++)
```

```
            for (k = 0; k < z; k++)
```

```
                scanf("%d", &B[i][j][k]);
```

```

// Addition

printf("\n3D Matrix Addition Result:\n");

for (i = 0; i < x; i++) {
    printf("Layer %d:\n", i + 1);
    for (j = 0; j < y; j++) {
        for (k = 0; k < z; k++) {
             $C[i][j][k] = A[i][j][k] + B[i][j][k];$ 
            printf("%d ", C[i][j][k]);
        }
        printf("\n");
    }
}

```

```

// Subtraction

printf("\n3D Matrix Subtraction Result:\n");

for (i = 0; i < x; i++) {
    printf("Layer %d:\n", i + 1);
    for (j = 0; j < y; j++) {
        for (k = 0; k < z; k++) {
             $C[i][j][k] = A[i][j][k] - B[i][j][k];$ 
            printf("%d ", C[i][j][k]);
        }
        printf("\n");
    }
}

```

```

// Multiplication (layer-wise)

printf("\n3D Matrix Multiplication Result (layer-wise):\n");

for (i = 0; i < x; i++) {

```

```

        printf("Layer %d:\n", i + 1);
        for (j = 0; j < y; j++) {
            for (k = 0; k < z; k++) {
                C[i][j][k] = A[i][j][k] * B[i][j][k];
                printf("%d ", C[i][j][k]);
            }
            printf("\n");
        }
    }

    return 0;
}

```

The screenshot shows a terminal window with the following output:

```

Output
Enter dimensions (x y z): 2 2 2
Enter elements of 3D Matrix A:
1 2
3 4
5 6
7 8
Enter elements of 3D Matrix B:
1 1
1 1
1 1
1 1
3D Matrix Addition Result:
Layer 1:
2 3
4 5
Layer 2:
6 7
8 9
3D Matrix Subtraction Result:
Layer 1:
0 1
2 3
Layer 2:
4 5
6 7
3D Matrix Multiplication Result (layer-wise):
Layer 1:
1 2
3 4
Layer 2:
5 6
7 8
Press Enter to continue...

```

EXP NO 16: C PROGRAM TO IMPLEMENT A SINGLE LINKED LIST

```
// EXP16_SingleLinkedList.c
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

struct Node {
    int data;
    struct Node *next;
};

```

```

int main() {

    struct Node *head = NULL, *temp, *newnode;

    int n, value;

    printf("Enter number of nodes: ");

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {

        newnode = (struct Node *)malloc(sizeof(struct Node));

        printf("Enter data for node %d: ", i + 1);

        scanf("%d", &value);

        newnode->data = value;

        newnode->next = NULL;

        if (head == NULL)

            head = newnode;

        else {

            temp = head;

            while (temp->next)

                temp = temp->next;

            temp->next = newnode;

        }

    }

    printf("Linked List: ");

    temp = head;

    while (temp) {

        printf("%d -> ", temp->data);

        temp = temp->next;

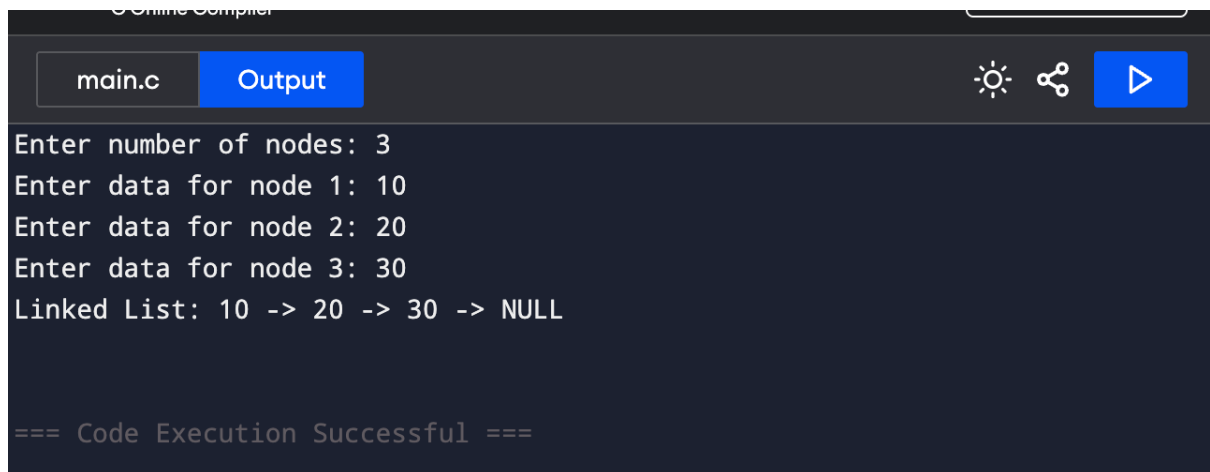
    }

    printf("NULL\n");

    return 0;
}

```

```
}
```



```
main.c Output
Enter number of nodes: 3
Enter data for node 1: 10
Enter data for node 2: 20
Enter data for node 3: 30
Linked List: 10 -> 20 -> 30 -> NULL

=== Code Execution Successful ===
```

EXP NO17: WRITE A C PROGRAM TO IMPLEMENT STACK OPERATION IN ARRAY[PUSH]

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX], top = -1;
```

```
void push(int val) {
```

```
    if (top == MAX - 1)
```

```
        printf("Stack Overflow\n");
```

```
    else {
```

```
        top++;
```

```
        stack[top] = val;
```

```
        printf("%d pushed into stack\n", val);
```

```
    }
```

```
}
```

```
int main() {
```

```
    int val;
```

```
    printf("Enter value to push: ");
```

```
    scanf("%d", &val);
```

```
    push(val);  
    return 0;  
}
```

Output Clear

Enter value to push: 25
25 pushed into stack

=== Code Execution Successful ===

EXP NO18: WRITE A C PROGRAM TO IMPLEMENT STACK OPERATION IN ARRAY [POP]

```
#include <stdio.h>  
  
#define MAX 5  
  
int stack[MAX] = {10, 20, 30}, top = 2;  
  
void pop() {  
    if (top == -1)  
        printf("Stack Underflow\n");  
    else  
        printf("Popped element: %d\n", stack[top--]);  
}  
  
int main() {  
    pop();  
    return 0;  
}
```

```
Online Compiler

main.c Output

Popped element: 30

=== Code Execution Successful ===
```

EXP NO19: WRITE A C PROGRAM TO IMPLEMENT STACK OPERATION IN ARRAY[DISPLAY]

```
#include <stdio.h>

#define MAX 5

int stack[MAX] = {10, 20, 30}, top = 2;

void display() {
    if (top == -1)
        printf("Stack is empty\n");
    else {
        printf("Stack elements: ");
        for (int i = top; i >= 0; i--)
            printf("%d ", stack[i]);
        printf("\n");
    }
}

int main() {
    display();
    return 0;
}
```



```
Online Compiler

main.c Output

Stack elements: 30 20 10

=== Code Execution Successful ===
```

EXP NO20: WRITE A C PROGRAM TO IMPLEMENT STACK OPERATION IN LINKED LIST[PUSH]

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

```
struct Node *top = NULL;
```

```
void push(int val) {  
    struct Node *newnode = (struct Node *)malloc(sizeof(struct Node));  
    newnode->data = val;  
    newnode->next = top;  
    top = newnode;  
    printf("%d pushed to stack\n", val);  
}
```

```
int main() {  
    int val;  
    printf("Enter value to push: ");
```

```

scanf("%d", &val);
push(val);
return 0;
}

```

EXP NO21: WRITE A C PROGRAM TO IMPLEMENT STACK OPERATION IN LINKED LIST[POP]

// EXP21_StackPop_LinkedList.c

#include <stdio.h>

#include <stdlib.h>

```

struct Node {
    int data;
    struct Node *next;
};

```

struct Node *top = NULL;

```

void push(int val) {
    struct Node *newnode = (struct Node *)malloc(sizeof(struct Node));
    newnode->data = val;
    newnode->next = top;
    top = newnode;
}

```

```
void pop() {  
    if (top == NULL)  
        printf("Stack Underflow\n");  
    else {  
        struct Node *temp = top;  
        printf("Popped element: %d\n", temp->data);  
        top = top->next;  
        free(temp);  
    }  
}
```

```
int main() {  
    push(10);  
    push(20);  
    push(30);  
    pop();  
    return 0;  
}
```

```
// EXP21_StackPop_LinkedList.c
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

```
struct Node *top = NULL;
```

```
void push(int val) {  
    struct Node *newnode = (struct Node *)malloc(sizeof(struct Node));  
    newnode->data = val;  
    newnode->next = top;  
    top = newnode;  
}
```

```
void pop() {  
    if (top == NULL)  
        printf("Stack Underflow\n");  
    else {  
        struct Node *temp = top;  
        printf("Popped element: %d\n", temp->data);  
        top = top->next;  
        free(temp);  
    }  
}
```

```
int main() {  
    push(10);  
    push(20);  
    push(30);  
    pop();  
    return 0;  
}
```

```
Online Compiler

main.c Output

Popped element: 30

=== Code Execution Successful ===
```

EXP NO22: WRITE A C PROGRAM TO IMPLEMENT STACK OPERATION IN LINKED LIST[DISPLAY]

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node *next;
};
```

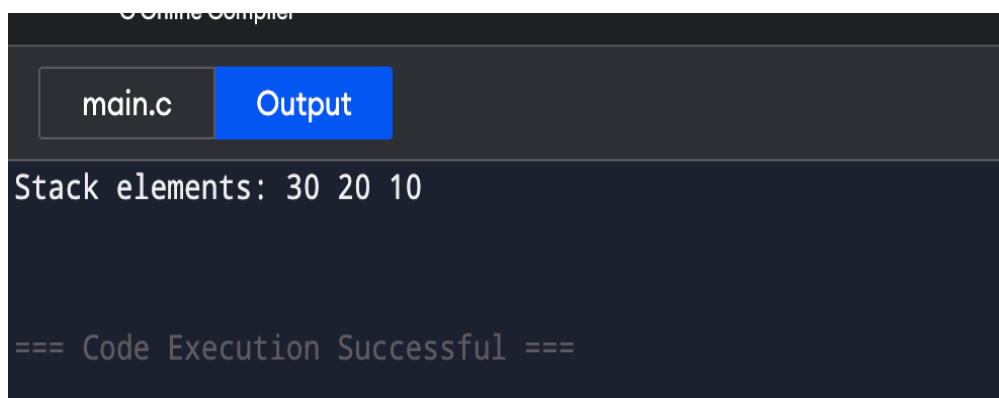
```
struct Node *top = NULL;
```

```
void push(int val) {
    struct Node *newnode = (struct Node *)malloc(sizeof(struct Node));
    newnode->data = val;
    newnode->next = top;
    top = newnode;
}
```

```
void display() {
    if (top == NULL)
        printf("Stack is empty\n");
    else {
```

```
    struct Node *temp = top;
    printf("Stack elements: ");
    while (temp) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

```
int main() {
    push(10);
    push(20);
    push(30);
    display();
    return 0;
}
```



The screenshot shows a dark-themed online compiler interface. At the top, there's a header bar with a logo and the text "Online Compiler". Below this, there are two tabs: "main.c" and "Output". The "Output" tab is active, showing the program's output. The output text is "Stack elements: 30 20 10" followed by "=== Code Execution Successful ===" on a new line.